OPERATING SYSTEM (CT 656)

TU | IOE | BCT | Year III / Part II

ALL NUMERICAL QUESTIONS — UNIQUE TYPES ONLY

One best representative question per type | Full step-by-step solutions | 2070–2082 Coverage

Contents: §1 CPU Scheduling §2 Page Replacement §3 Disk Scheduling §4 Banker's Algorithm §5 Memory Address §6 Deadlock Detection

SECTION 1 | CPU SCHEDULING

Q1 2080 Ashwin [[5]]

Find the average waiting time and average turnaround time using: FCFS, SRTN, RR (TQ = 2)
Process P1 P2 P3 P4 Arrival 0 1 4 6 Burst 3 6 4 2

PART A — FCFS (First Come First Served)

```
| P1(0–3) | P2(3–9) | P3(9–13) | P4(13–15) |
```

Process	AT	BT	CT	TAT = CT−AT	WT = TAT−BT
P1	0	3	3	3	0
P2	1	6	9	8	2
P3	4	4	13	9	5
P4	6	2	15	9	7

✓ **Avg TAT = (3+8+9+9)/4 = 7.25 ms Avg WT = (0+2+5+7)/4 = 3.5 ms**

PART B — SRTN (Shortest Remaining Time Next = Preemptive SJF)

→ At each new arrival, compare its burst with current remaining burst; preempt if shorter.

t=0: P1(rem=3) runs. t=1: P2(6) arrives, P1 rem=2 < 6 → continue P1.
t=3: P1 done. P2(6) only → run P2. t=4: P3(4) arrives, P2 rem=5 > 4 → preempt, run P3.
t=6: P4(2) arrives, P3 rem=2 = P4 rem=2 → tie, keep P3. t=8: P3 done. P4(2) shorter than P2(5) → run P4.
t=10: P4 done. P2 rem=5 → run P2. t=15: P2 done.

```
| P1(0–3) | P2(3–4) | P3(4–8) | P4(8–10) | P2(10–15) |
```

Process	AT	BT	CT	TAT	WT
P1	0	3	3	3	0
P2	1	6	15	14	8
P3	4	4	8	4	0
P4	6	2	10	4	2

✓ **Avg TAT = (3+14+4+4)/4 = 6.25 ms Avg WT = (0+8+0+2)/4 = 2.5 ms**

PART C — RR (Round Robin, Q = 2 ms)

→ At each quantum expiry, if remaining > 0 → rejoin rear of queue. New arrivals during a quantum join immediately.

t=0–2: P1 runs (rem=1). P2 arrived@1→queued. Queue=[P2,P1].
t=2–4: P2 runs (rem=4). P3@4→queued. Queue=[P1,P3,P2].

t=4–5: P1 done (rem=1<Q). CT(P1)=5. Queue=[P3,P2].
t=5–7: P3 runs (rem=2). P4@6→queued. Queue=[P2,P4,P3].
t=7–9: P2 runs (rem=2). Queue=[P4,P3,P2].
t=9–11: P4 done (rem=2). CT(P4)=11. Queue=[P3,P2].
t=11–13: P3 done (rem=2). CT(P3)=13. Queue=[P2].
t=13–15: P2 done (rem=2). CT(P2)=15.

```
| P1(0–2) | P2(2–4) | P1(4–5) | P3(5–7) | P2(7–9) | P4(9–11) | P3(11–13) | P2(13–15) |
```

Process	AT	BT	CT	TAT	WT
P1	0	3	5	5	2
P2	1	6	15	14	8
P3	4	4	13	9	5
P4	6	2	11	5	3

✓ **Avg TAT = (5+14+9+5)/4 = 8.25 ms Avg WT = (2+8+5+3)/4 = 4.5 ms**

Q2 2081 Chaitra [[8]]

Schedule using HRRN (Highest Response Ratio Next) and calculate Avg WT, Avg TAT. Process A B C D Arrival 0 2 5 10 Burst 12 8 7 9

□ HRRN — Response Ratio = (WT + BT) / BT → pick HIGHEST ratio; NON-preemptive

t=0: Only A available → run A (BT=12). A finishes at t=12.
t=12: Ready = B(WT=10,BT=8,RR=18/8=2.25), C(WT=7,BT=7,RR=14/7=2.00), D(WT=2,BT=9,RR=11/9≈1.22)
Highest RR = B → run B. Finishes t=20.
t=20: Ready = C(WT=15,BT=7,RR=22/7≈3.14), D(WT=10,BT=9,RR=19/9≈2.11)
Highest RR = C → run C. Finishes t=27.
t=27: Only D → run D. Finishes t=36.

```
| A(0–12) | B(12–20) | C(20–27) | D(27–36) |
```

Process	AT	BT	CT	TAT	WT
A	0	12	12	12	0
B	2	8	20	18	10
C	5	7	27	22	15
D	10	9	36	26	17

✓ **Avg TAT = (12+18+22+26)/4 = 19.5 ms Avg WT = (0+10+15+17)/4 = 10.5 ms**

Q3 2082 Kartik [[7]]

Schedule using SRTN AND Preemptive Priority. Lowest priority number = highest priority. Process P0 P1 P2 P3 P4 Arrival 0 2 4 8 12 Burst 8 2 12 4 10 Priority 2 4 0 3 1

□ PREEMPTIVE PRIORITY (lower number = higher priority)

P2 has priority 0 (highest). P4 has priority 1 (second).
t=0: P0(pri=2) only → run P0.
t=2: P1(pri=4) arrives. P0(2) higher → continue P0.

t=4: P2(pri=0) arrives → PREEMPT P0 (P0 ran 4ms, rem=4). Run P2.
t=8: P3(pri=3) arrives. P2(0) highest → continue P2.
t=12: P4(pri=1) arrives. P2(0) highest → continue P2.
t=16: P2 done. Ready: P0(rem=4,pri=2), P1(BT=2,pri=4), P3(BT=4,pri=3), P4(BT=10,pri=1). Highest=P4(1) → run P4.
t=26: P4 done. Highest=P0(2) → run P0. t=30: P0 done. Highest=P3(3) → run P3.
t=34: P3 done. Run P1. t=36: P1 done.

```
| P0(0–4) | P2(4–16) | P4(16–26) | P0(26–30) | P3(30–34) | P1(34–36) |
```

Process	AT	BT	CT	TAT	WT
P0	0	8	30	30	22
P1	2	2	36	34	32
P2	4	12	16	12	0
P3	8	4	34	26	22
P4	12	10	26	14	4

✓ **Avg TAT = (30+34+12+26+14)/5 = 23.2 ms Avg WT = (22+32+0+22+4)/5 = 16.0 ms**

Q4 2071 Bhadra [[2+3+5]]

Schedule using Multilevel Feedback Queue. Three queues: Q1 (RR q=5 ms), Q2 (RR q=10 ms), Q3 (FCFS) Processes join Q1 initially; demote to next queue if not done. Process P1 P2 P3 P4 Arrival 0 12 25 32 Burst 25 18 4 10

□ MLFQ — Q1(RR,q=5) → Q2(RR,q=10) → Q3(FCFS)

All new processes enter Q1. If not finished after 1 quantum → demote to Q2. If not done in Q2 → Q3.
t=0–5: Q1: P1 runs 5ms (rem=20). Demote P1 to Q2. Q1=[], Q2=[P1].
t=5–15: Q1 empty. Q2: P1 runs 10ms (rem=10). Demote P1 to Q3. Q2=[], Q3=[P1].
t=12: P2 arrives → Q1=[P2]. (P1 currently running in Q2 to t=15)
t=15–20: Q1: P2 runs 5ms (rem=13). Demote P2 to Q2. Q1=[], Q2=[P2].
t=20–25: Q2: P2 runs 5ms (rem=8). P3@25 arrives → Q1=[P3]. Serve Q1 first.
t=25–29: Q1: P3 runs 4ms (done). CT(P3)=29. Q1=[], Q2=[P2].
t=29–32: Q2: P2 runs 3ms (rem=5). P4@32 → Q1=[P4]. Serve Q1.
t=32–37: Q1: P4 runs 5ms (done). CT(P4)=37. Back to Q2=[P2].
t=37–42: Q2: P2 runs 5ms (done). CT(P2)=42. Q3=[P1].
t=42–52: Q3: P1 runs 10ms (done). CT(P1)=52.

```
| P1(0–5) | P1(5–15) | P2(15–20) | P2(20–25) | P3(25–29) | P2(29–32) | P4(32–37) |
P2(37–42) | P1(42–52) |
```

Process	AT	BT	CT	TAT	WT
P1	0	25	52	52	27
P2	12	18	42	30	12
P3	25	4	29	4	0
P4	32	10	37	5	0

✓ **Avg TAT = (52+30+4+5)/4 = 22.75 ms Avg WT = (27+12+0+0)/4 = 9.75 ms**

SECTION 2 | PAGE REPLACEMENT ALGORITHMS

Q5 2082 Kartik [[6]]

Consider the following page reference strings: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 Find number of page hits for FIFO, Optimal and LRU — Assume available frames = 4

PART A — FIFO (First In First Out)

→ Replace the page that entered memory EARLIEST.

Page	1	2	3	4	1	2	5	1	2	3	4	5
Frame1	1	1	1	1	1	1	2	3	4	5	1	2
Frame2	-	2	2	2	2	2	3	4	5	1	2	3
Frame3	-	-	3	3	3	3	4	5	1	2	3	4
Frame4	-	-	-	4	4	4	5	1	2	3	4	5
Fault?	F	F	F	F	H	H	F	F	F	F	F	F

✓ **FIFO: Faults = 10, Hits = 2**

PART B — OPTIMAL (Replace page NOT used for longest time in future)

Page	1	2	3	4	1	2	5	1	2	3	4	5
Frame1	1	1	1	1	1	1	1	1	1	1	2	2
Frame2	-	2	2	2	2	2	2	2	2	2	3	3
Frame3	-	-	3	3	3	3	3	3	3	3	4	4
Frame4	-	-	-	4	4	4	5	5	5	5	5	5
Fault?	F	F	F	F	H	H	F	H	H	H	F	H

✓ **OPTIMAL: Faults = 6, Hits = 6 (minimum possible)**

PART C — LRU (Replace Least Recently Used page)

Page	1	2	3	4	1	2	5	1	2	3	4	5
Frame1	1	1	1	1	1	1	5	5	5	3	4	5
Frame2	-	2	2	2	2	2	1	1	1	1	1	2
Frame3	-	-	3	3	3	3	2	2	2	2	2	3
Frame4	-	-	-	4	4	4	4	4	4	5	3	4
Fault?	F	F	F	F	H	H	F	H	H	F	F	F

✓ **LRU: Faults = 8, Hits = 4**

Q6 2080 Chaitra [[1+6]]

What is Demand Paging? Consider page reference string: 2, 3, 4, 2, 1, 3, 7, 5, 4, 3, 1, 5 — 3 page frames Find page faults for OPTIMAL, LRU, LFU (Least Frequently Used).

LFU — Replace page with LOWEST access frequency; tie→LRU

Page	2	3	4	2	1	3	7	5	4	3	1	5
Frame1	2	2	2	2	2	2	2	2	2	2	2	2
Frame2	-	3	3	3	3	3	3	3	3	3	3	3
Frame3	-	-	4	4	1	1	7	5	4	4	1	5
Fault?	F	F	F	H	F	H	F	F	F	H	F	F

✓ **LFU: Faults = 9, Hits = 3**

→ Demand Paging: Pages loaded only when needed (on demand). Page fault = page not in memory → OS loads it from disk.

Q7 2073 Magh [[3+2]]

Show Belady's Anomaly with FIFO. Use page reference: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 Compare faults with 3 frames vs 4 frames.

□ FIFO with 3 frames

Ref	1	2	3	4	1	2	5	1	2	3	4	5
F1	1	1	1	2	3	4	1	1	1	2	5	3
F2	-	2	2	3	4	1	2	2	2	5	3	4
F3	-	-	3	4	1	2	5	5	5	3	4	5
?	F	F	F	F	F	F	F	H	H	F	F	H

✓ **3 frames → Faults = 9**

▶ FIFO with 4 frames (from Q5 above → Faults = 10)

From Q5 Part A: 4 frames gave Faults = 10.

3 frames → 9 faults 4 frames → 10 faults ← MORE faults with MORE frames!

✓ **This is Belady's Anomaly: with FIFO, increasing frames can INCREASE page faults. Doesn't occur with LRU/Optimal.**

Q8 2072 Ashwin [[3+6]]

Consider a paged memory system with 8 pages of 8KB page size each and 16 page frames in memory. Page table: Page 7→10, 6→4, 5→0, 4→7, 3→13, 2→11, 1→14, 0→5 Compute the physical address for logical address 18325.

□ Physical Address = Frame × Page_Size + Offset

Page size = 8 KB = 8192 bytes = 2^{13}

Logical address = 18325

Page number = □18325 / 8192□ = □2.23□ = 2

Offset = 18325 mod 8192 = 18325 − 16384 = 1941

Frame for page 2 = 11 (from page table above)

Physical address = 11 × 8192 + 1941 = 90112 + 1941 = 92053

→ Logical address bits: page number bits = $\log_2$ (8) = 3, offset bits = $\log_2$ (8192) = 13 → 16-bit logical address

---

SECTION 3 | DISK SCHEDULING ALGORITHMS

Q9 2080 Ashwin [[2+8]]

Suppose a disk drive has 2000 cylinders (0–1999). Drive currently at cylinder 134, previous request at 124. Pending queue (FIFO): 86, 1470, 913, 1774, 948, 1509, 1022, 1750, 130 Calculate total head movement for: (i) FCFS (ii) SSTF (iii) SCAN (iv) C-SCAN (v) LOOK

Setup: Head at 134, Previous at 124 (moving in INCREASING direction for SCAN)

→ Queue: 86, 1470, 913, 1774, 948, 1509, 1022, 1750, 130 | Head = 134 | Direction = toward 1999

i) FCFS — Service in request order

134→86→1470→913→1774→948→1509→1022→1750→130

Move	From→To	Distance
1	134→86	48
2	86→1470	1384
3	1470→913	557
4	913→1774	861
5	1774→948	826
6	948→1509	561
7	1509→1022	487
8	1022→1750	728
9	1750→130	1620

✓ **FCFS Total = 48+1384+557+861+826+561+487+728+1620 = 7072 cylinders**

ii) SSTF — Always service the NEAREST request

From 134: nearest=130(4). From 130: nearest=86(44). From 86: nearest=913(827). From 913: nearest=948(35).

From 948: nearest=1022(74). From 1022: nearest=1470(448). From 1470: nearest=1509(39). From 1509: nearest=1750(241).

From 1750: nearest=1774(24).

Order: 134→130→86→913→948→1022→1470→1509→1750→1774

✓ **SSTF Total = 4+44+827+35+74+448+39+241+24 = 1736 cylinders**

iii) SCAN (Elevator) — Sweep to end, then reverse

Direction = increasing (toward 1999). Serve requests on the way up, then sweep back.

134→ [go up] →130? No, 130<134. Sort ascending above 134: 913,948,1022,1470,1509,1750,1774 → hit end(1999) → reverse

Going UP: 134→913→948→1022→1470→1509→1750→1774→1999

Coming DOWN: →130→86

✓ **SCAN Total = (1999−134)+(1999−86) = 1865+1913 = ... = (913−134)+(948−913)+...**

Up: 134→913(779)→948(35)→1022(74)→1470(448)→1509(39)→1750(241)→1774(24)→1999(225)

Down: 1999→130(1869)→86(44)

✓ **SCAN Total = 779+35+74+448+39+241+24+225+1869+44 = 3778 cylinders**

iv) C-SCAN — Sweep to end, jump to 0, continue same direction

Go UP: 134→913→948→1022→1470→1509→1750→1774→1999→0→86→130

Distances: 779+35+74+448+39+241+24+225+(1999 jump to 0)+86+44

✓ **C-SCAN Total = 779+35+74+448+39+241+24+225+1999+86+44 = 3994 cylinders**

v) LOOK — Like SCAN but only goes to last request, not end of disk

Sorted above 134: 913,948,1022,1470,1509,1750,1774 Below 134: 130,86
Go UP to 1774 (last): 134→913→948→1022→1470→1509→1750→1774
Reverse DOWN to 86: 1774→130→86
Distances: 779+35+74+448+39+241+24 + (1774−86) = 1640+1688

✓ **LOOK Total = 779+35+74+448+39+241+24+1644+44 = 3328 cylinders**

Q10 2078 Chaitra [[2+6]]

> Disk has 100 cylinders (0–99). Head at 43, previous at 25. Pending queue: 86, 70, 13, 74, 48, 9, 22, 50, 30 Calculate for: FCFS, C-SCAN, SSTF

Setup: Head=43, Direction=increasing (43>25 so moving up)

FCFS

43→86→70→13→74→48→9→22→50→30
Moves: 43+16+57+61+26+39+13+28+20 = 303

✓ **FCFS = 303 cylinders**

SSTF

43: nearest=48(5). 48: nearest=50(2). 50: nearest=70(20). 70: nearest=74(4). 74: nearest=86(12).
86: nearest=30(56). 30: nearest=22(8). 22: nearest=13(9). 13: nearest=9(4).
43→48→50→70→74→86→30→22→13→9

✓ **SSTF = 5+2+20+4+12+56+8+9+4 = 120 cylinders**

C-SCAN

Go up: 43→48→50→70→74→86→99(end)→0→9→13→22→30
Moves: 5+2+20+4+12+13(to 99) + 99(jump) + 9+4+9+8

✓ **C-SCAN = 5+2+20+4+12+13+99+9+4+9+8 = 185 cylinders**

SECTION 4 | BANKER'S ALGORITHM (Deadlock Avoidance)

Q11 2082 Kartik / 2081 Chaitra (Most Repeated) [[2+8]]

> 5 concurrent processes (P0–P4), 4 resources (R0–R3). Total instances: R0=3, R1=17, R2=16, R3=12 Allocation: Maximum Claim: R0 R1 R2 R3 R0 R1 R2 R3 P0: 0 1 1 0 0 2 1 0 P1: 1 2 3 1 1 6 5 2 P2: 1 3 6 5 2 3 6 6 P3: 0 6 3 2 0 6 5 2 P4: 0 0 1 4 0 6 5 6 a) Create Need matrix b) Is system safe? Show safe sequence.

Step 1 — Calculate NEED = Maximum − Allocation

Process	Max R0	Max R1	Max R2	Max R3	Alloc R0	Alloc R1	Alloc R2	Alloc R3	Need R0	Need R1	Need R2	Need R3
P0	0	2	1	0	0	1	1	0	0	1	0	0
P1	1	6	5	2	1	2	3	1	0	4	2	1
P2	2	3	6	6	1	3	6	5	1	0	0	1

P3	0	6	5	2	0	6	3	2	0	0	2	0
P4	0	6	5	6	0	0	1	4	0	6	4	2

□ Step 2 — Calculate AVAILABLE = Total − Sum(Allocation)

Sum of all allocations: R0=0+1+1+0+0=2, R1=1+2+3+6+0=12, R2=1+3+6+3+1=14, R3=0+1+5+2+4=12

Available = [3−2, 17−12, 16−14, 12−12] = [1, 5, 2, 0]

□ Step 3 — Safety Algorithm (Work = Available = [1,5,2,0])

Step	Process	Need ≤ Work?	Work After (if yes)	Status
1	P0	[0,1,0,0]≤[1,5,2,0] ✓	[1+0,5+1,2+1,0+0]=[1,6,3,0]	FINISH
2	P3	[0,0,2,0]≤[1,6,3,0] ✓	[1+0,6+6,3+3,0+2]=[1,12,6,2]	FINISH
3	P2	[1,0,0,1]≤[1,12,6,2] ✓	[1+1,12+3,6+6,2+5]=[2,15,12,7]	FINISH
4	P1	[0,4,2,1]≤[2,15,12,7] ✓	[2+1,15+2,12+3,7+1]=[3,17,15,8]	FINISH
5	P4	[0,6,4,2]≤[3,17,15,8] ✓	[3+0,17+0,15+1,8+4]=[3,17,16,12]	FINISH

✓ SYSTEM IS SAFE ✓

✓ Safe Sequence: P0 → P3 → P2 → P1 → P4

Q12 2080 Ashwin [[4+6]]

5 processes (P0–P4), 3 resource types (A, B, C). Available = [1, 0, 1] Allocation: Maximum: A B C A B C P0: 0 1 0 7 5 3 P1: 2 0 0 3 2 2 P2: 3 0 2 9 0 2 P3: 2 1 1 2 2 2 P4: 0 0 2 4 3 3 a) Is state safe? If so, safe sequence. b) P1 requests [1,0,1] — can it be granted?

□ Step 1 — Need Matrix

Process	Alloc A	Alloc B	Alloc C	Max A	Max B	Max C	Need A	Need B	Need C
P0	0	1	0	7	5	3	7	4	3
P1	2	0	0	3	2	2	1	2	2
P2	3	0	2	9	0	2	6	0	0
P3	2	1	1	2	2	2	0	1	1
P4	0	0	2	4	3	3	4	3	1

□ Step 2 — Safety Check (Work = Available = [1,0,1])

Check each process: Need[i] ≤ Work?

P0: [7,4,3]≤[1,0,1]? NO. P1: [1,2,2]≤[1,0,1]? NO (B:2>0). P2: [6,0,0]≤[1,0,1]? NO.

P3: [0,1,1]≤[1,0,1]? NO (B:1>0). P4: [4,3,1]≤[1,0,1]? NO.

No process can run with Work=[1,0,1]. Try after recheck...

P3 Need=[0,1,1]. Work=[1,0,1]. B needed=1, available=0 → still NO.

> → Re-reading: Available given as [1,0,1]. Let me check which can proceed... P1 Need=[1,2,2]: B=2>0=Avail. None passes first round.

This means: UNSAFE STATE — deadlock possible. No safe sequence exists with Available=[1,0,1].

(If Available were [3,3,2], safe sequence would be P1→P3→P4→P0→P2 — standard textbook example)

□ Part b) P1 requests [1,0,1]

Check: Request ≤ Need? [1,0,1]≤[1,2,2] ✓

Check: Request ≤ Available? [1,0,1]≤[1,0,1] ✓

Pretend to allocate: Available=[0,0,0], Alloc(P1)=[3,0,1], Need(P1)=[0,2,1]

Run safety: Work=[0,0,0]. No process can run → UNSAFE → DENY REQUEST.

✓ **P1's request [1,0,1] must be DENIED — granting would lead to unsafe state.**

Q13 2079 Chaitra [[5]]

Identify unsafe vs deadlock state. Process Has/Max, Free Resource = 2: Process Has Max A 2 6 B 1 5 C 2 3 D 3 8 a) What happens if Process A requests 1 resource? b) What happens if Process D requests 1 resource?

Setup: Total Allocated=2+1+2+3=8, Free=2, Total=10

Need = Max − Has: A=4, B=4, C=1, D=5

Part a) A requests 1 resource (Free becomes 1)

After granting: Free=1, Has(A)=3, Need(A)=3

Safety check with Free=1:

C: Need=1 ≤ Free=1 → run C → free 2 → Free=3

A: Need=3 ≤ 3 → run A → Free=6

B: Need=4 ≤ 6 → run B → Free=7

D: Need=5 ≤ 7 → run D → safe!

✓ **Grant A's request. Safe sequence: C→A→B→D**

Part b) D requests 1 resource (Free becomes 1)

After granting: Free=1, Has(D)=4, Need(D)=4

Safety check with Free=1:

C: Need=1 ≤ 1 → run C → free 2 → Free=3

A: Need=4 > 3. B: Need=4 > 3. D: Need=4 > 3. NO process can proceed!

✓ **DENY D's request. Granting leads to UNSAFE STATE → potential deadlock.**

SECTION 5 | MEMORY MANAGEMENT (Address, Best/Worst/First Fit)

Q14 2079 Chaitra [[5]]

We have memory of 1000 KB with 5 partitions: 200 KB, 400 KB, 600 KB, 500 KB, 300 KB (in order) Processes A (175 KB) and B (400 KB) need to be loaded. Which partitions are chosen by Best-Fit and Worst-Fit?

Best-Fit — Choose SMALLEST partition that still fits

Process	Size	Best-Fit Partition	Why
A	175 KB	200 KB	Smallest that fits 175KB (200>175, 300>175, 400>175, 500>175, 600>175 → smallest = 200)
B	400 KB	400 KB	After A uses 200KB partition, remaining: 400,600,500,300. Smallest ≥ 400 = 400KB

Worst-Fit — Choose LARGEST partition

Process	Size	Worst-Fit Partition	Why

A	175 KB	600 KB	Largest available partition
B	400 KB	500 KB	After A in 600KB, largest remaining = 500KB

✓ **Best-Fit minimizes wasted space. Worst-Fit leaves large holes for future large processes.**

Q15 2080 Ashwin [[5]]

What is Inode? Consider a 20-GB disk with 8-KB block size. How much memory will be occupied if contiguous allocation is used for file allocation? Assume file allocation table (FAT) entry takes 4 bytes.

Calculation

Total disk size = 20 GB = 20 × 1024 × 1024 KB = 20,971,520 KB
Block size = 8 KB
Number of blocks = 20,971,520 / 8 = 2,621,440 blocks
Each FAT entry = 4 bytes
Total FAT size = 2,621,440 × 4 bytes = 10,485,760 bytes = 10 MB
For contiguous allocation: only need start block + length per file → small directory entry.

→ Inode: Index node storing file metadata (size, owner, timestamps, pointers to data blocks). Superior to FAT because: no traversal needed, direct+indirect+double-indirect block pointers, file size only limited by disk size.

SECTION 6 | DEADLOCK DETECTION (Multi-instance Resources)

Q16 2077 Chaitra [[3+7]]

5 processes (P0–P4), 3 resource types A(7 inst), B(2 inst), C(6 inst). At time t0: Allocation: Request: A B C A B C P0: 0 1 0 0 0 0 P1: 2 0 0 2 0 2 P2: 3 0 3 0 0 0 P3: 2 1 1 1 0 0 P4: 0 0 2 0 0 2 Available = A:0, B:0, C:0 a) Is state safe? b) P2 requests [0,0,1] — what effect?

Available = [0,0,0] — Detection Algorithm (like safety, use Request not Need)

Work=[0,0,0]. Find process where Request≤Work and Finish=false.
P0: Request=[0,0,0]≤[0,0,0] ✓ → run P0. Work=[0+0,0+1,0+0]=[0,1,0]. Finish(P0)=true.
P1: Request=[2,0,2]≤[0,1,0]? 2>0 → NO.
P2: Request=[0,0,0]≤[0,1,0] ✓ → run P2. Work=[0+3,1+0,0+3]=[3,1,3]. Finish(P2)=true.
P3: Request=[1,0,0]≤[3,1,3] ✓ → run P3. Work=[3+2,1+1,3+1]=[5,2,4]. Finish(P3)=true.
P4: Request=[0,0,2]≤[5,2,4] ✓ → run P4. Work=[5+0,2+0,4+2]=[5,2,6]. Finish(P4)=true.
P1: Request=[2,0,2]≤[5,2,6] ✓ → run P1. Work=[5+2,2+0,6+0]=[7,2,6]. Finish(P1)=true.

✓ **All processes finish → NO DEADLOCK. Sequence: P0→P2→P3→P4→P1**

Part b) P2 requests [0,0,1]

Check: Request ≤ Available? [0,0,1]≤[0,0,0]? NO (C: 1 > 0)

✓ **DENY immediately — not enough available resources. P2 must wait.**

QUICK REFERENCE SUMMARY

Topic	Key Formula / Rule	Common Mistake
FCFS	CT = previous CT + BT (order = arrival)	Ignores burst time completely
SJF (Non-preemptive)	At CPU free time → pick shortest BT in ready queue	Does NOT preempt mid-execution
SRTN / SRTF	On every arrival → compare new BT with remaining; preempt if new is shorter	Compare with REMAINING, not original BT
HRRN	RR = (WT+BT)/BT. Non-preemptive. Recalculate at each CPU free slot	WT changes as you recalculate
Priority	Lower number = higher priority (usually). Preemptive: check on every arrival	Starvation → fix with Aging
RR	Maintain queue; new arrival joins REAR; if not done rejoin REAR after quantum	Track REMAINING burst carefully
FIFO Page	Replace page in FIFO order (a queue). Subject to Belady's Anomaly	Belady's only affects FIFO, not LRU/OPT
LRU Page	Replace page not used for longest time. Count backward from current.	Must track last-use time per page
Optimal Page	Replace page with farthest next use (or never-used)	Unrealizable in practice; used as benchmark
LFU Page	Replace page with lowest access count; tie → LRU	Reset counts periodically to avoid old pages staying
FCFS Disk	Serve in arrival order. Simple but worst performance	Total = sum of |consecutive differences|
SSTF Disk	Nearest cylinder first. Good throughput but starvation possible	Always recompute nearest from current position
SCAN Disk	Go to end, reverse. Like elevator. No starvation	Goes all the way to end even if no requests there
LOOK Disk	Like SCAN but stops at last request, not disk end	More efficient than SCAN — standard in practice
C-SCAN	One-directional sweep; jump back to 0	Adds full disk jump distance (0 to max)
Banker's Need	Need = Maximum − Allocation	Always check Need, not Maximum
Banker's Safety	Work=Available; find Need≤Work; add Allocation to Work; repeat	All processes must finish → safe sequence
Request Grant	Request≤Need AND Request≤Available → pretend allocate → safety test	Must satisfy BOTH conditions AND pass safety

Best of Luck! | TU IOE BCT | Operating System (CT 656) | All Unique Numerical Types